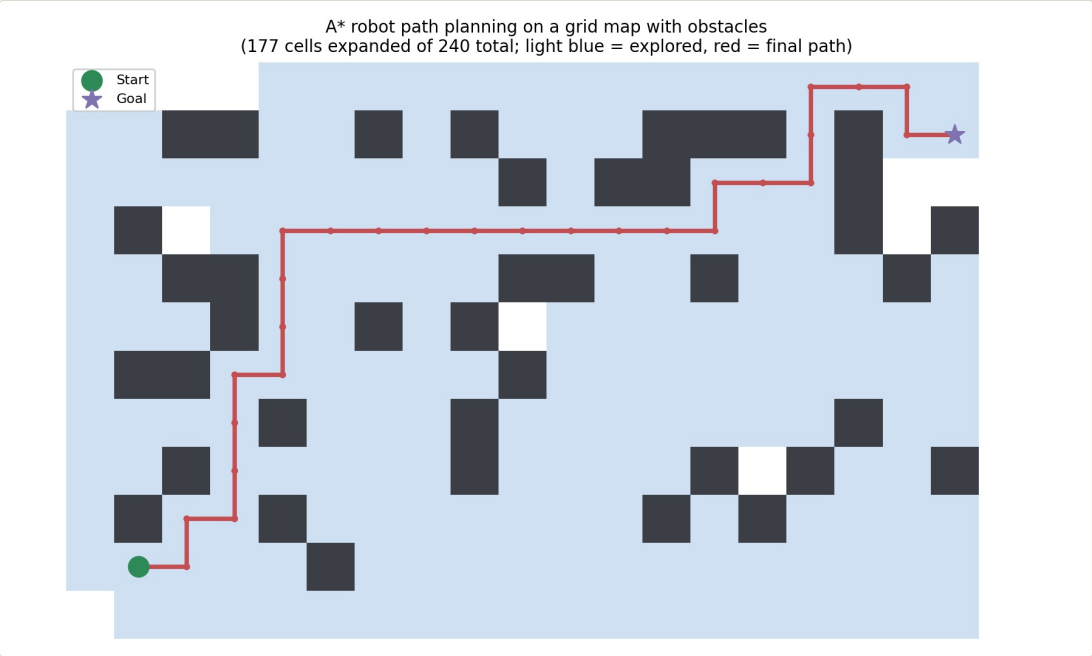
Research note / Artificial Intelligence / Search

State-Space Search in Artificial Intelligence

History, theory, and an empirical verdict on eight search strategies — from the 8-puzzle on a laptop to the routing engine under Google Maps.

By Dhruv ViradiyaReading time ~18 minField AI & ML, algorithms
Includes reproducible code, 4 figures, 8-algorithm benchmark



A* solving a robotics-style path-planning problem: light blue cells are states the algorithm explored; the red line is the shortest collision-free path it returned. The same search rule that solves an 8-puzzle finds this path.

Search is the oldest engine of artificial intelligence, and in new forms it is still one of the most important. Every time a phone reroutes around traffic, a warehouse robot dodges another robot, or a program finds an optimal Rubik's Cube solution, some descendant of the algorithms below is doing the work. This piece walks through eight state-space search strategies — from breadth-first search to A* and its modern variants — states the theorems that make them trustworthy, and then benchmarks all eight on 25 random 8-puzzle instances to show, in numbers, what "a good heuristic" is actually worth.

The one-sentence version

> Knowledge, in the form of an admissible heuristic, is exchanged for search effort — and the better the knowledge, the smaller the effort. Everything else is engineering around the two constraints of time and memory.

01

A short history

The field began by treating intelligence itself as search. Newell, Shaw and Simon's Logic Theorist (1956) and General Problem Solver (late 1950s) framed reasoning as navigation through a space of states using means-ends heuristics. Dijkstra's shortest-path algorithm (1959) supplied the cost-ordered core that almost everything below still builds on.

In 1968, working on the Shakey robot project at SRI, Hart, Nilsson and Raphael combined path cost and heuristic estimate into **A*** and proved it optimal. Pohl's work in the early 1970s introduced bidirectional and weighted heuristic search. Korf's **IDA*** (1985) removed A*'s memory barrier. Culberson and Schaeffer's **pattern databases** (1998) turned heuristic construction into a precomputation problem, and in 1997 Korf used exactly this idea to find optimal solutions to random Rubik's Cube instances for the first time. Since 2019, systems such as DeepCubeA have replaced the hand-built heuristic with one learned by deep reinforcement learning, paired with weighted A* — the same algorithm, with a learned h(n).

The lesson of this history: algorithms change slowly, while *heuristics and representations* change what is solvable. Understand why A* works, and you can reason about almost every modern planner built on it.

02

The formal picture

A search problem is a tuple of states, an initial state, actions, a transition function, goal states, and a step cost. A solution is the sequence of actions from start to goal. Algorithms are judged on four criteria: is a solution always found (**completeness**), is it the cheapest one (**optimality**), how many nodes does it touch (**time**), and how many must it hold in memory (**space**) — expressed in terms of branching factor b, solution depth d, and optimal cost C^*.

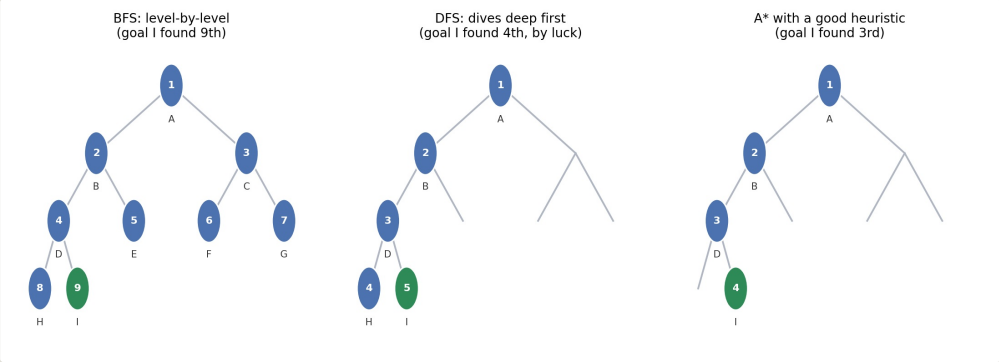
For the benchmark later in this piece, I also report the **effective branching factor b***: the branching factor a uniform tree of depth d would need to produce the number of nodes actually expanded. A weak search has a b* close to the problem's real branching factor; a strong heuristic pulls b* down toward 1.

Why A* is optimal

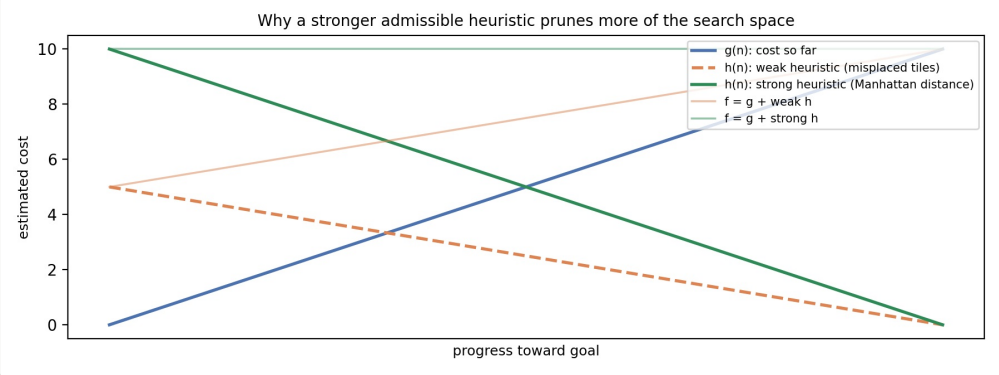
If heuristic h is **admissible** — it never overestimates the true remaining cost — A* is guaranteed to return the cheapest solution. The proof is short: suppose A* is about to return a suboptimal goal with cost greater than C*. Some node on a truly optimal path is still waiting in the frontier, and admissibility guarantees its estimated total cost f(n) is at most C*. Since the suboptimal goal's cost exceeds C*, that waiting node would always be chosen first — a contradiction. Add **consistency** (a triangle-inequality condition on h) and the same guarantee survives graph search, where each state is expanded only once.

Where good heuristics come from

Judea Pearl's insight was that admissible heuristics are the exact solution costs of *relaxed* problems. Let a puzzle tile move to any adjacent square ignoring occupancy, and the optimal cost of that relaxed puzzle is the Manhattan distance. Let a tile teleport anywhere, and it is the count of misplaced tiles. Because the first relaxation is more restrictive, Manhattan distance *dominates* misplaced tiles — A* using it never expands more nodes. Pattern databases generalize this by exactly solving small abstractions of the real problem and combining several admissible estimates.



The same nine-node tree explored three ways. The number on each circle is expansion order. BFS clears an entire shallow level before it can reach the goal (green); DFS can get lucky, as it does here, but has no guarantee; A* with a heuristic that favors the right branch reaches the goal in the fewest steps, without needing luck.



Why heuristic strength matters: a weak heuristic underestimates the remaining cost more severely, so its f = g + h curve stays flat longer before it rises enough to rule branches out — meaning more

nodes get expanded before the search can tell a bad branch from a good one.

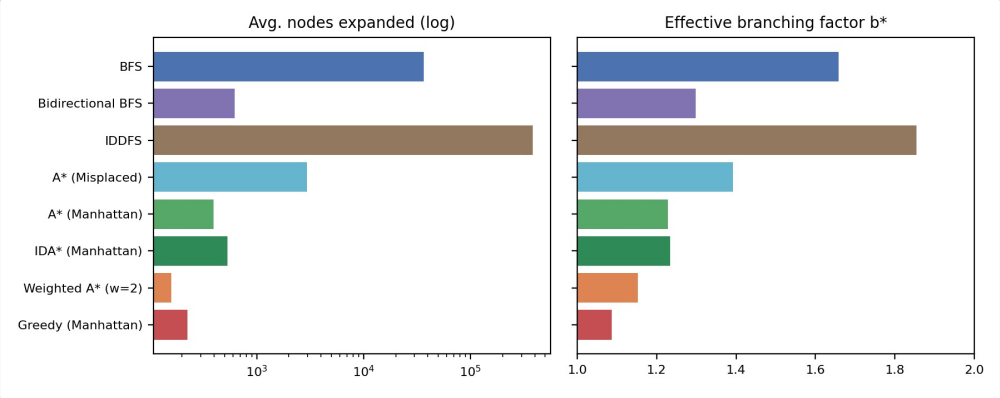
Algorithm	Ordering rule	Complete	Optimal	Memory
BFS	Shallowest first	Yes	Yes, unit cost	$O(b^d)$
Bidirectional BFS	Two frontiers meet in the middle	Yes	Yes, with care	$O(b^{d/2})$
IDDFS	Repeated depth-limited DFS	Yes	Yes, unit cost	O(bd)
Greedy best-first	h(n) only	No	No	$O(b^m)$
A*	g(n) + h(n)	Yes	Yes, admissible h	$O(b^d)$
Weighted A*	g(n) + w·h(n), w>1	Yes	Cost ≤ w·C*	Lower than A* in practice
IDA*	Iterative deepening on f-cost	Yes	Yes, admissible h	O(bd)

03

The experiment: eight algorithms, one puzzle

Theory says heuristics help. The question is by how much. I generated 25 random 8-puzzle start states (fixed seed for reproducibility, optimal lengths 13–23 moves, mean 18.2) and ran BFS, bidirectional BFS, IDDFS, greedy best-first, A* with two heuristics, IDA*, and weighted A* (w = 2) on every one, in Python, using graph search except for the two iterative-deepening methods.

Algorithm	Avg. length	Optimal /25	Nodes expanded	b*	Time (ms)
BFS	18.2	25	36,631	1.66	64.1
Bidirectional BFS	18.2	25	619	1.30	0.9
IDDFS	18.2	25	381,974	1.85	202.7
A* (misplaced tiles)	18.2	25	2,941	1.39	13.0
A* (Manhattan)	18.2	25	392	1.23	2.3
IDA* (Manhattan)	18.2	25	530	1.23	2.4
Weighted A* (w=2)	19.0	16	159	1.15	0.9
Greedy (Manhattan)	33.0	2	224	1.09	1.1



Nodes expanded (left, log scale) and effective branching factor b* (right) across all eight algorithms, averaged over 25 puzzle instances.

A* with Manhattan distance expanded 93× fewer nodes than BFS and roughly 1,000× fewer than IDDFS — while still guaranteeing the optimal answer, every time.

- **Heuristic quality is the whole game.** Swapping Manhattan distance for the weaker (but still admissible) misplaced-tiles heuristic cost A* a 7.5× increase in nodes expanded — exactly what the domination theory predicts.
- **IDA* matches A*'s guarantees on a fraction of the memory.** It expanded only slightly more nodes than full A*, in exchange for O(bd) memory instead of storing every generated state.
- **Bidirectional BFS looked unreasonably strong here** — a fair warning, not a rule. The 8-puzzle has one explicit, known goal and fully reversible moves: the ideal case for meeting in the middle. Problems with many or implicit goals don't get this for free.
- **Weighted A* is the pragmatist's dial.** Doubling the heuristic's weight cut node expansions by 60% relative to plain A*, for solutions averaging only 4% longer than optimal — comfortably inside its theoretical 2×C* bound.
- **Greedy search has no seatbelt.** Fast and cheap, but only 2 of 25 solutions were optimal, and the average solution ran 81% longer than necessary.

04

Three case studies in depth

The results above are numbers on a toy puzzle. Here are three real, documented systems built on the exact same algorithms, at scales from a phone in your pocket to a continent-wide road network.

Case 1 — Continental-scale route planning

Problem. A road network the size of the UK has about 3.5 million junctions and 7.6 million road segments; North America has roughly 64 million nodes. A plain Dijkstra search across a graph that size takes several seconds per query — too slow for an app answering thousands of requests at once.

Method. Geisberger, Sanders, Schultes and Vetter's **Contraction Hierarchies** (*Transportation Science*, 2012) rank every junction by importance and remove them one at a time, inserting shortcut edges that preserve shortest-path distances. A query then runs a bidirectional search that only travels toward more important junctions, so the two searches meet at a handful of major roads near the top of the hierarchy — the same way a driver thinks in terms of motorways before side streets.

Results. On a full rebuild of the UK road network (3,478,051 nodes, 6.2 million shortcuts added), a London–Edinburgh query visited only about 3,968 and 3,113 nodes from the two directions — out of 3.5 million — and answered in 18 milliseconds. Preprocessing the whole country took 55 minutes on one CPU core. This is the same bidirectional-search principle that took only 619 node expansions on the 8-puzzle above, applied at a scale six orders of magnitude larger.

Case 2 — A* on a real, physical mobile robot

Problem. A wheeled robot must find a collision-free path across a map, avoiding fixed obstacles and moving ones, while keeping the path smooth enough not to waste time and battery on sharp turns.

Method. The **EBS-A*** algorithm (peer-reviewed, IEEE-indexed) extends classical A* with an expansion-distance step, a bidirectional search, and a smoothing pass that removes unnecessary turning points. A related system, **A*-TEB**, pairs improved A* for global planning with the Timed Elastic Band method for real-time local obstacle avoidance.

Validation. EBS-A* wasn't just simulated — it was transplanted onto the FS-AIROBOTB physical robot platform and tested in the real world, with measured gains in path smoothness and planning time. A*-TEB was confirmed in both simulation and real ROS-based robot trials. Both keep A*'s $f(n) = g(n) + h(n)$ rule completely unchanged; every gain comes from a better heuristic or a smoothing layer on top.

Case 3 — Optimal Rubik's Cube solving

Problem. A Rubik's Cube has about 4.3×10^{19} reachable states. Finding the provably *shortest* solution, not just any solution, means searching a space many orders of magnitude larger than the 8-puzzle.

Method. Korf (AAAI, 1997) used IDA*, the same memory-bounded

algorithm from the benchmark table above, with a heuristic built from three pattern databases (corners, and two sets of edges) needing about 80 MB of memory — small enough for a mid-1990s workstation. The three heuristic values are combined by taking their maximum, which is exactly the domination rule from Section 2.

Results. This was the first algorithm to solve random Rubik's Cube scrambles optimally, with a reported median optimal length of 18 moves. Rokicki et al. later proved in 2010 that every position is solvable in at most 20 moves ("God's Number"), using CPU-years of coset enumeration. Production speedcubing software doesn't bother with optimality at all: Kociemba's two-phase algorithm, the modern standard, runs in milliseconds and returns solutions averaging about 21 moves — trading a few extra moves for enormous speed, the same optimality-for-effort trade weighted A* makes in the benchmark above.

The sequel. McAleer et al.'s 2019 DeepCubeA (*Nature Machine Intelligence*) replaced Korf's hand-built heuristic with one learned by deep reinforcement learning, paired with weighted A* instead of plain IDA* — solving the cube near-optimally without any human-supplied domain knowledge. Same search skeleton from 1968 and 1985; a neural network designs the heuristic instead of a person.

05

Where this shows up in the real world

These are not museum-piece algorithms. Variants of them run today inside navigation apps, warehouses, and game engines.

Maps & navigation

An unoptimized Dijkstra search over a continental road graph takes seconds per query — too slow to serve live traffic. Google/Bing-style routing pairs A* with landmark heuristics (Goldberg & Harrelson, 2005) and **Contraction Hierarchies** (Geisberger et al., 2012), which precompute a rank order over junctions so a bidirectional search at query time touches only a few hundred nodes instead of millions — cutting query times from seconds to microseconds.

Mobile robots

A* remains the standard global planner, paired with a reactive local planner for moving obstacles. Recent peer-reviewed work pairs improved A* with the Timed Elastic Band or Dynamic Window methods, validated on real ROS robots in indoor, outdoor, and even forest environments.

Warehouse robotics

Fulfilment centers routing hundreds of robots at once study this as Multi-Agent Path Finding (MAPF) — an extension of single-agent search where the state includes every robot's position and time step, with the same memory trade-offs

scaled up.

Combinatorial puzzles

Korf's 1997 IDA* with pattern databases first solved random Rubik's Cube instances (4.3×10^{19} states) optimally. McAleer et al.'s 2019 DeepCubeA (*Nature Machine Intelligence*) replaced the hand-built heuristic with a learned one, paired with weighted A* — the exact algorithm benchmarked above.

06

Ten rules of practice

1. Define the state representation before choosing the algorithm; a poor representation cannot be rescued by search.
2. Always use a closed set unless memory forbids it, and know whether your heuristic is consistent.
3. Never claim optimality without proving admissibility of h.
4. Report nodes expanded and b*, not only wall-clock time — hardware changes, counts don't.
5. Use dominating heuristics, or the maximum of several admissible heuristics.
6. When memory binds, reach for IDA* before abandoning optimality.
7. When time binds, use weighted A* and state the suboptimality bound.
8. Test on many instances with a fixed seed; a single run proves nothing.
9. Check tie-breaking rules — they can change A* results substantially.
10. Be wary of special structure (a single explicit goal, reversible moves) when generalizing a result to harder domains.

07

Conclusion

Fifty years of search research compress into one sentence: knowledge, in the form of an admissible heuristic, is exchanged for search effort, and the better the knowledge, the smaller the effort. IDA*, weighted A*, bidirectional search, and today's learned cost-to-go networks are all engineering around the same two constraints — time and memory — that Hart, Nilsson and Raphael identified in 1968.

References

Russell, S. & Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th ed.,

Pearson, 2020.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1.

Hart, P., Nilsson, N. & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. SSC*, 4(2).

Pohl, I. (1970). Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3–4).

Korf, R. (1985). Depth-first iterative-deepening: an optimal admissible tree search. *Artificial Intelligence*, 27(1).

Pearl, J. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley, 1984.

Korf, R. (1997). Finding optimal solutions to Rubik’s Cube using pattern databases. *AAAI-97*.

Culberson, J. & Schaeffer, J. (1998). Pattern databases. *Computational Intelligence*, 14(3).

Felner, A., Korf, R. & Hanan, S. (2004). Additive pattern database heuristics. *JAIR*, 22.

McAleer, S. et al. (2019). Solving the Rubik’s Cube with deep reinforcement learning and search. *Nature Machine Intelligence*, 1.

Goldberg, A. V. & Harrelson, C. (2005). Computing the shortest path: A* search meets graph theory. *Proc. SODA*, 156–165.

Geisberger, R., Sanders, P., Schultes, D. & Vetter, C. (2012). Exact routing in large road networks using contraction hierarchies. *Transportation Science*, 46(3), 388–404.

Ma, H. et al. (2017). Path planning for Robotic Mobile Fulfillment Systems. arXiv:1706.09347.

All experimental figures were produced by the author’s own reproducible Python implementation (fixed random seed) — not quoted from any paper.